

Claims Agent — Conversation Handover

Project: C:\claims_bot_with_agent

Purpose: continue building the orchestrator for the PhilHealth claims workflow on a different PC.

Python: 3.13.7 at C:\claims_bot_with_agent\.venv\Scripts\python.exe

Read this first. The conversation mixed three kinds of statement, and some early answers were design sketches presented too confidently. This document labels every claim:

- **VERIFIED** — written to disk and run; output shown.
- **PARTIAL** — file exists and imports, but not exercised end-to-end.
- **ILLUSTRATIVE** — shown in chat as a sketch only; never written to disk or executed. Not working code.

1. What you asked for

- Build an AI agent like **Hermes**, tailored to your own PhilHealth claims workflow.
- The agent should observe your manual work, learn it, and eventually apply what it learned automatically.
- Trigger should be a **button**.
- Output should be an **Excel report** showing the status of each claim that was processed.

2. Your workflow, as you described it

#	Step	Status today
1	Scan documents → scan folder; existing script renames PDFs and extracts patient name, hospital number, confinement period	Exists
2	Processed claims move to <code>output\</code> , one folder per patient	Exists
3	Fees Check → reports what is missing, writes a report	Exists
4a	Missing dates → Date Fill button, log into HBSys, entry point, type hospital number, click Date Fill; then Fees Check again	Exists
4b	Missing final bill → done manually	Does not exist
5	eClaims → some manual clicking, then XML clicker button generates CF4, CF5, eSOA	Exists
6	Background script copies XML from shared FTP folder into the patient folder	Exists
7	Check Missing → complete folders to READY, incomplete to INCOMPLETE	Exists
8	Upload clicker button attaches files; transmit manually	Exists

Of eight steps, only **final bill** lacks an automation. That is the one genuinely new tool you will need.

3. The "agent/" package — read this with care

This package is not the answer to your question. It was built early, before we discovered you already have deterministic tools for every step.

It records screen clicks and typing coordinates and "learns" by matching action-type sequences. It was never run against a real HBSys session, and it cannot supply a patient name it does not already have — the exact gap you identified when asking "paano nya malalaman ang name ng patient?" That question is answered in section 4, and the answer is the folder name, not a learner.

File	State	Note
agent/observation.py	PARTIAL	Records clicks/typing. Unit-tested only.
agent/learning.py	PARTIAL	Extracts patterns by exact action-type sequence. Brittle: a reordered click/click/TYPE sequence is treated as a different pattern.
agent/memory.py	PARTIAL	JSON persistence.
agent/automation.py	PARTIAL	Plays back actions. <code>dry_run=True</code> is the default, which is the correct safety posture.
agent/agent_core.py	PARTIAL	Orchestrates the above.
agent/gui.py	VERIFIED	Loads and builds. One real bug found and fixed here — section 6.1.
agent/test_agent.py	VERIFIED	6 tests passing.
agent_cli.py	PARTIAL	Subcommands: <code>status</code> , <code>observe</code> , <code>learn</code> , <code>gui</code> .

4. How the agent knows which patient

This was your question, and asking it was the right move — it exposed that the `agent/` package could never answer it.

The patient is **not** looked up in a database and **not** typed by hand. The folder name is the record. Every output folder is already named:

```
GABAY, MERCEDES BACANI - 00000000021627 - ADM20260913_DIS20260916
+--- name +---+ +--- hospital no --+ +--- confinement --+
```

That is the format produced by `core/patient_folder_naming.py::build_patient_folder_name()`. Parsing that one string yields the hospital number, patient name and confinement period — exactly the three values the HBSys clickers need — with no database write and no hardcoded patient data. It also keeps your project rule that database access stays read-only.

VERIFIED — `core/pipeline_context.py` implements this:

```
cd /d C:\claims_bot_with_agent
.venv\Scripts\python.exe core\pipeline_context.py
```

All 14 checks pass. It parses the real folder name from your `fees_checker_report.csv` and **refuses** — never guesses — short hospital numbers, names without a comma, and missing ADM/DIS dates.

5. Why the decision layer is rules, not learning

The problem vocabulary is already stable and machine-produced, so a learner adds risk without adding value:

- `fees_checker.ready_to_generate_xml()` returns a fixed problem list, e.g.
`hprofserv.pdoctorsigndate BLANK , ADM date mismatch .`
- `claims_checker` returns a `Missing` column of document kinds: CSF, SOA1, COE, DTR, MRF, PBC...

These are exact strings. An exact-match rule table is simpler, faster and testable. A fuzzy learner could mis-route a claim; a rule table cannot.

Escalation principle. Anything not explicitly mapped is **HUMAN**. A missing tool must never degrade into "click something and hope".

Missing / problem	Action	Why
<code>hpatcon1.consentdate BLANK</code>	<code>date_fill_regular</code>	Date Fill clicker
<code>hprofserv.pdoctorsigndate BLANK</code>	<code>date_fill_regular</code>	Date Fill clicker
<code>hpatcon1.authsigndate BLANK</code>	<code>date_fill_regular</code>	Date Fill clicker
ADM / DIS date mismatch	<code>date_fill_regular</code>	Date Fill corrects them
CF4 / CF5 / eSOA XML missing	<code>xml_clicker</code>	XML generator
Status is MISMATCH (fees)	HUMAN	Itemized and grouped charges disagree — billing review
CSF / SOA1 / COE / DTR / MRF missing	HUMAN	Documents cannot be generated
Unrecognised problem	HUMAN	Never guessed

ABTC claims route to the separate `date_fill_abtc` node rather than the regular date-fill node.

6. Bugs found and fixed (VERIFIED)

6.1 GUI crash — `tag_config`

Running `python agent_cli.py gui` raised:

```
AttributeError: 'Treeview' object has no attribute 'tag_config'.  
Did you mean: 'tag_configure'?
```

Fixed by renaming the call to `tag_configure`. The GUI then loaded and built without error.

6.2 Hardcoded paths broken by the rename — this one mattered

You renamed `C:\claims_bot` to `C:\claims_bot_with_agent`, but three files still pointed at the old path. Confirmed by probe:

```
fees_checker OUTPUT_DIR: C:\claims_bot\output    <-- does not exist
```

Both checkers had `BASE_DIR = Path(r"C:\claims_bot")` hardcoded. Fixed in `fees_checker.py` and `claims_checker.py` to resolve from the file's own location, with an env-var override:

```
BASE_DIR = Path(
    os.environ.get("CLAIMS_BASE_DIR") or Path(__file__).resolve().parent
)
```

`claims_gui_config.json` was also repointed from `C:\claims_bot\...\`` to `C:\claims_bot_with_agent\...` for `base_dir`, `scan_folder`, `output_folder`, `backup_folder`, `review_staging_folder`, `signature_folder` and `sqlite_db`.

Why this matters for the other PC: because paths now resolve relative to the file, the project folder can be renamed or moved again without breaking anything – as long as the whole folder is copied together.

7. Work in progress – the orchestrator

Not finished. Do not wire this into the GUI yet. `core/pipeline_orchestrator.py` was being written when the conversation ended, and its test suite was mid-debug.

It contains: per-patient iteration over `output\`; a check → decide → run tool → re-check loop; the outcomes `READY`, `FIXED`, `HUMAN`, `FAILED`, `REFUSED`; a `dry_run=True` default; per-patient exception isolation; a `max_fix_rounds` guard against a tool that never clears the problem; and an Excel run report.

The last test run had 7 failures, all of which turned out to be **test-harness bugs, not code bugs**: the stub problem-source was shared across all patients in one temp directory, so state leaked between them and every count was doubled or tripled. The fix was to give each scenario its own isolated directory. That refactor was in progress when the session ended, so **re-run the suite before trusting the module**:

```
cd /d C:\claims_bot_with_agent
.venv\Scripts\python.exe -m core.pipeline_orchestrator
```

Only `core/pipeline_context.py` is confirmed green (14/14).

8. What still has to be built

Priority	Item	Why
1	Finish + green the orchestrator test suite	Cannot build on an unverified module

2	Final bill clicker (new tool)	The only step with no automation. Needs the exact HBSys click sequence from you.
3	Wire orchestrator to a GUI button	You asked for a button trigger
4	Wire the Excel run report to the GUI	You asked for per-claim status
5	Decide on HBSys login: auto or manual?	You said "maglologin ako" – unclear if login can be automated
6	Decide batch scope: one patient per run, or all folders?	Affects the safety clamp

9. Questions you asked, and the answers

Kaya mo ba gawan ng AI agent gaya ng Hermes pero tailored sa workflow ko?

Yes – but the useful version turned out to be an orchestrator over your existing tools, not a click-learner. See section 5.

Paano kung sa ibang task, like generating monthly Annex A, B, C – kayang matutunan?

It can be *automated*, yes, but "learned from observation" is the fragile way. Annex generation should be a **deterministic script** driven by a date and an output folder, exactly like the other steps. That is more reliable than inferring a click sequence.

Kailangan ba maghintay ng tatlong buwan bago magawa ang skill?